

WWW Studio — Design Intelligence Module

Complete Agent Development Roadmap

Branch: `feature/design-intelligence`

Status: Ready to implement

Depends on: Phase 0–9 complete (agent_notes.md current as of 2026-06-22)

Overview & Architecture Decisions

Before the agent writes a single line of code, read and understand these decisions — they govern every implementation choice in this roadmap.

What this feature is

A **multi-source design synthesis engine** that accepts a primary website URL, optional secondary URLs, and annotated image references, then produces a complete design package: `design.md`, `tailwind.config.ts`, `tokens.css`, and `design-tokens.json`. The output is live-editable before download, and can be applied directly to the current project's design panel.

Where it lives in the stack

- **New API route:** `artifacts/api-server/src/routes/design-extract.ts`
- **New DB table:** `design_extractions` (Drizzle schema in `packages/db`)
- **New frontend page:** `artifacts/www-studio/src/pages/DesignExtract.tsx` at route `/design-extract`
- **New frontend components:** `artifacts/www-studio/src/components/design-extract/` (subdirectory)
- **Screenshot service:** Reuse and extend `artifacts/api-server/src/routes/screenshot-to-code.ts` pattern
- **LLM calls:** Always via `artifacts/api-server/src/lib/llm.ts` — never create OpenAI clients inline

- **Static frontend AI path:** For GitHub Pages deployment, implement a direct Gemini API path in the frontend (same pattern as `AiChatWidget.tsx`)

Critical architecture note

The project runs in two modes:

1. **GitHub Pages (static)** — frontend calls Gemini API directly using user's key from `localStorage`
2. **Self-hosted with api-server** — frontend calls `/api/design-extract/*` , which uses `llm.ts`

The Design Extract page **must support both modes** with the same triple-fallback pattern used in `SceneChat.tsx` .

Phase A: Database & API Foundation

Branch checkpoint: `feature/design-intel/phase-a`

A1 — Drizzle Schema: `design_extractions` table

File: `packages/db/src/schema.ts` (add to existing schema)

```
// Add this table definition
export const designExtractions = pgTable('design_extractions', {
  id: uuid('id').defaultRandom().primaryKey(),
  userId: text('user_id').notNull().default('guest'),
  projectId: uuid('project_id').references(() => projects.id, { onDelete: 'set nu

// Input
primary_url: text('primary_url').notNull(),
references: jsonb('references').notNull().default('[]'),
// references shape: Array<{
//   id: string,
//   type: 'url' | 'image',
//   value: string,           // URL string or base64 data URI
//   filename?: string,      // for images
//   annotation: string,      // user's note e.g. "love this font"
//   intent?: string,         // parsed intent: 'typography'|'color'|'layout'|'v
//   weight?: number,         // 0-1, influence weighting derived from annotatio
// }>

// Extraction status
```

```

status: text('status').notNull().default('pending'),
// pending | processing | complete | failed
error: text('error'),
processingTimeMs: integer('processing_time_ms'),

// Output – raw strings stored, parsed on frontend
outputDesignMd: text('output_design_md'),
outputTailwindConfig: text('output_tailwind_config'),
outputTokensCss: text('output_tokens_css'),
outputDesignTokensJson: text('output_design_tokens_json'),

// Extracted design data as structured JSONB for live editing
extractedTokens: jsonb('extracted_tokens').default('{}'),
// shape: {
//   colors: { primary, secondary, accent, background, surface, text, muted, bo
//   typography: { displayFont, bodyFont, monoFont, scaleBase, weights }[],
//   spacing: { base, scale }[],
//   radii: { sm, md, lg, full }[],
//   shadows: string[],
//   animations: string[],
//   brandRationale: string,
//   overview: string,
// }

// Metadata
createdAt: timestamp('created_at').defaultNow().notNull(),
updatedAt: timestamp('updated_at').defaultNow().notNull(),
savedToKb: boolean('saved_to_kb').default(false),
});

```

After adding: run `pnpm --filter @workspace/db run push` to apply migration.

A2 — Screenshot Service Extension

File: `artifacts/api-server/src/lib/screenshot.ts` (new file)

Create a dedicated screenshot utility that the design-extract route and the existing screenshot-to-code route can both import.

```

// Exports:
export async function screenshotUrl(url: string): Promise<string>
// Returns base64 PNG string
// Strategy (try in order):
//   1. If SCREENSHOTONE_KEY env var present → call ScreenshotOne API
//   2. If PLAYWRIGHT_ENDPOINT env var present → call remote Playwright

```

```
// 3. Fallback: return placeholder base64 (log warning)

export async function screenshotUrlWithViewports(url: string): Promise<{
  desktop: string; // base64, 1440px wide
  mobile: string; // base64, 390px wide
  tablet: string; // base64, 768px wide
}>
```

Implementation notes:

- ScreenshotOne free tier: 100 screenshots/month — sufficient for personal use
- Env var: `SCREENSHOTONE_KEY` — document in README
- The `screenshotUrlWithViewports` function is used specifically for secondary URL references so we can honor annotations like "use mobile layout from this site"
- Always return something — never throw; log warnings and return placeholder if all services fail

A3 — Intent Parser Utility

File: `artifacts/api-server/src/lib/intentParser.ts` (new file)

This utility interprets user annotations into structured extraction directives.

```
export type ReferenceIntent =
  | 'typography' // "use this font", "love this typeface"
  | 'color' // "love this colour", "this palette", "this vibe"
  | 'layout' // "use this layout", "grid structure"
  | 'mobile-layout' // "use mobile layout from this"
  | 'animation' // "love these animations", "movement style"
  | 'blend' // "mix between these", "combine both"
  | 'base' // "use as base", "primary reference"
  | 'inspiration' // "i love this vibe", "this aesthetic"
  | 'component' // "use these cards/buttons"

export function parseAnnotationIntent(annotation: string): {
  intent: ReferenceIntent;
  weight: number; // 0.0-1.0 influence weight
  sections: string[]; // which design.md sections to pull from: 'colors'|'typogr
  directive: string; // human-readable instruction for the LLM prompt
}
```

Pattern match on keywords (case-insensitive):

- font|typeface|typography → intent: typography, sections: ['typography']
 - colour|color|palette|vibe|aesthetic → intent: color, sections: ['colors']
 - mobile layout|mobile nav|mobile menu → intent: mobile-layout, sections: ['layout'],
add mobile qualifier
 - mix|combine|blend|both → intent: blend, weight: 0.5 for both sources
 - base|primary|foundation → intent: base, weight: 1.0
 - animation|motion|movement|transition → intent: animation, sections: ['animations']
 - Default → intent: inspiration, weight: 0.7, sections: ['colors', 'typography', 'layout']
-

A4 — Design Extraction LLM Prompt System

File: artifacts/api-server/src/lib/designPrompts.ts (new file)

```
export function buildExtractionSystemPrompt(): string
// Returns the system prompt for the extraction LLM call
// Instructs the model to:
//   - Output ONLY valid JSON matching the extractedTokens schema
//   - Extract 6-10 colors with semantic names
//   - Identify all font families (display, body, mono)
//   - Extract spacing scale and border radii
//   - Write a 2-sentence brand rationale
//   - Note any unique design signatures

export function buildExtractionUserPrompt(params: {
  primaryUrl: string;
  primaryScreenshots: { desktop: string; mobile: string };
  references: Array<{
    type: 'url' | 'image';
    screenshots?: { desktop: string; mobile: string };
    imageBase64?: string;
    annotation: string;
    parsedIntent: ReturnType<typeof parseAnnotationIntent>;
  }>;
}): MessageContent[]
// Returns the messages array for visionComplete()
// Includes all screenshots as image blocks
// Per-reference: states the annotation and parsed directive explicitly
// Example directive injection:
//   "Reference 2 (annotation: 'use mobile layout from this'):"
//   Extract ONLY the mobile navigation and responsive layout patterns.
```

```
//      Apply these specifically to the Layout/Mobile section of the output."

export function buildDesignMdFromTokens(tokens: ExtractedTokens, primaryUrl: stri
// Converts the structured JSON tokens into the full design.md document
// Follows the canonical 8-section structure:
//   1. Overview, 2. Colors, 3. Typography, 4. Layout,
//   5. Elevation & Depth, 6. Shapes, 7. Components, 8. Do's and Don'ts
// Includes YAML front matter with machine-readable tokens

export function buildTailwindConfigFromTokens(tokens: ExtractedTokens): string
// Returns tailwind.config.ts file content as a string
// Extends theme with: colors, fontFamily, fontSize, borderRadius, boxShadow, spa

export function buildTokensCssFromTokens(tokens: ExtractedTokens): string
// Returns tokens.css with CSS custom properties
// :root { --color-primary: ...; --font-display: ...; etc. }

export function buildDesignTokensJsonFromTokens(tokens: ExtractedTokens): string
// Returns DTCG-format design-tokens.json
// Compatible with Style Dictionary, Tokens Studio, etc.
```

A5 — Main API Route

File: artifacts/api-server/src/routes/design-extract.ts (new file)

Register in artifacts/api-server/src/index.ts as:

```
import designExtractRouter from './routes/design-extract.js';
app.use('/api/design-extract', designExtractRouter);
```

Endpoints to implement:

POST /api/design-extract — Start extraction

```
Body: {
  primaryUrl: string,
  projectId?: string,
  references?: Array<{
    id: string,
    type: 'url' | 'image',
    value: string,          // URL or base64 data URI
    filename?: string,
    annotation: string,
```

```
}>
}
Response: { id: string, status: 'processing' }
```

- Validate `primaryUrl` (must be http/https)
- Parse all annotations with `parseAnnotationIntent()`
- Insert row into `design_extractions` with status: 'pending'
- Kick off async extraction pipeline (do NOT await — return immediately)
- Apply AI rate limiter middleware (30 req/min — same as other AI routes)

GET /api/design-extract/:id — Poll status + get result

```
Response: {
  id, status, error?,
  extractedTokens?,
  outputDesignMd?,
  outputTailwindConfig?,
  outputTokensCss?,
  outputDesignTokensJson?,
  processingTimeMs?
}
```

PATCH /api/design-extract/:id/tokens — Save live edits

```
Body: Partial<ExtractedTokens>
```

- Merge with existing `extractedTokens`
- Regenerate all output files from merged tokens
- Update `updatedAt`

POST /api/design-extract/:id/apply-to-project — Apply to project

```
Body: { projectId: string }
```

- Copy `extractedTokens` into the project's design panel data
- Update project's `designTokens` field (or equivalent in your schema)

GET /api/design-extract/:id/download/:format — Download file

Params: format = 'design-md' | 'tailwind-config' | 'tokens-css' | 'design-tokens-

- Returns file with correct content-type and Content-Disposition: attachment
- For 'zip': bundle all 4 files using `archiver` npm package

GET /api/design-extract — List user's past extractions

Query: ?limit=20&offset=0

Response: Array<{ id, primaryUrl, status, createdAt, extractedTokens.colors[0] }>

The async extraction pipeline (internal function):

```
async function runExtractionPipeline(extractionId: string) {
  const startTime = Date.now();

  try {
    // 1. Update status: processing
    await db.update(designExtractions).set({ status: 'processing' }).where(...)

    // 2. Screenshot primary URL
    const primaryShots = await screenshotUrlWithViewports(primaryUrl);

    // 3. Process each reference
    const processedRefs = await Promise.all(refs.map(async (ref) => {
      if (ref.type === 'url') {
        const shots = await screenshotUrlWithViewports(ref.value);
        return { ...ref, screenshots: shots, parsedIntent: parseAnnotationIntent(
        }
        // image type: value is already base64
        return { ...ref, parsedIntent: parseAnnotationIntent(ref.annotation) };
      }
    }));

    // 4. Build vision prompt
    const messages = buildExtractionUserPrompt({
      primaryUrl, primaryScreenshots: primaryShots, references: processedRefs
    });

    // 5. Call vision LLM
    const rawJson = await visionComplete(
      buildExtractionSystemPrompt(),
      messages,
      { responseFormat: 'json_object', maxTokens: 4000 }
    );
```

```

// 6. Parse + validate extracted tokens
const tokens = JSON.parse(rawJson) as ExtractedTokens;

// 7. Generate all output formats
const designMd = buildDesignMdFromTokens(tokens, primaryUrl);
const tailwindConfig = buildTailwindConfigFromTokens(tokens);
const tokensCss = buildTokensCssFromTokens(tokens);
const designTokensJson = buildDesignTokensJsonFromTokens(tokens);

// 8. Save complete result
await db.update(designExtractions).set({
  status: 'complete',
  extractedTokens: tokens,
  outputDesignMd: designMd,
  outputTailwindConfig: tailwindConfig,
  outputTokensCss: tokensCss,
  outputDesignTokensJson: designTokensJson,
  processingTimeMs: Date.now() - startTime,
}).where(...)

} catch (err) {
  await db.update(designExtractions).set({
    status: 'failed',
    error: err.message,
  }).where(...)
}
}

```

A6 — OpenAPI Spec Update

File: `artifacts/api-spec/openapi.yaml`

Add all `/design-extract` endpoints with full request/response schemas. After updating, run:

```
pnpm --filter @workspace/api-client-react run codegen
```

A7 — Rate Limiter Registration

In `artifacts/api-server/src/index.ts`, add `/api/design-extract` to the AI rate limiter middleware array (30 req/min). Add `/api/design-extract` base path to the general rate

limiter (300 req/15min).

Phase A checkpoint — verify before continuing:

- `pnpm run typecheck` passes with no errors
 - `pnpm run build` succeeds
 - `POST /api/design-extract` returns `{ id, status: 'processing' }` within 200ms
 - `GET /api/design-extract/:id` returns 'complete' within 30s for a simple URL
 - All 4 output files are non-empty and parseable
 - `GET /api/design-extract/:id/download/design-md` returns a downloadable file
-

Phase B: Frontend — Core Design Extract Page

Branch checkpoint: `feature/design-intel/phase-b`

B1 — Route Registration

File: `artifacts/www-studio/src/App.tsx`

Add:

```
<Route path="/design-extract" element={<DesignExtractPage />} />
<Route path="/design-extract/:id" element={<DesignExtractPage />} />
```

Add "Design Extract" to the Navbar with a `Pipette` or `Sparkles` icon from `lucide-react`.

B2 — Component File Structure

Create directory: `artifacts/www-studio/src/components/design-extract/`

Files to create (in order of implementation):

<code>DesignExtractInput.tsx</code>	— The main input area (primary URL + references)
<code>ReferenceItem.tsx</code>	— Single reference row (image or URL + annotation)
<code>ReferenceUploadPanel.tsx</code>	— Expanded panel for adding references
<code>ExtractionProgress.tsx</code>	— Progress indicator during processing
<code>DesignTokenEditor.tsx</code>	— Live-editable token display

ExportPanel	
-------------	--

B4 — DesignExtractInput Component

This is the centerpiece UI. Implement exactly as described in the feature spec:

Primary URL input:

```
// Full-width input with:
// - Placeholder: "Paste a website URL to extract its design..."
// - URL validation (must start with http/https) with inline error
// - Paste button (clipboard API) for quick paste on mobile
// - Clear button (×) when value present
// - Submit clears to "Extract Design" – no separate search button
```

References section (collapsed by default, expands on + click):

```
// Trigger:
<button onClick={() => setRefsOpen(true)}>
  + Add references
</button>
// Tooltip: "Upload images or add URLs with notes"
```

ReferenceUploadPanel (slide-in panel or inline accordion):

```
// Two tabs: "Image" | "URL"
//
// Image tab:
//   - Drag-and-drop zone + click to browse
//   - Accepts: jpg, png, webp, gif
//   - Max: 5MB per image, 5 images total
//   - Preview thumbnail on upload
//   - Annotation textarea: "Add a note about this reference..."
//   - Helper chips: ["Love this font" | "Use this colour" | "This vibe" | "Mobil
//     (clicking a chip appends that text to the annotation)
//
// URL tab:
//   - URL input (same validation as primary)
//   - Annotation textarea + helper chips
//   - Optional: auto-fetch favicon to show as thumbnail
```

ReferenceItem (shows after adding):

```
// Each reference shows as a row:
// [thumbnail] [type badge: URL/IMG] [annotation excerpt] [edit] [remove]
// Drag handle (dnd-kit) to reorder
// Click edit → expands back to edit form
```

Submit flow:

1. “Extract Design” button (disabled if no primary URL, loading spinner during submit)
 2. On click: call `POST /api/design-extract`, transition to `phase: 'processing'`
 3. Show character count of annotation (max 200 chars) for each reference
-

B5 — ExtractionProgress Component

Shows during `phase: 'processing'`. Make this feel alive, not like a dead spinner.

```
// Animated stages – rotate through these with 3s intervals:
const stages = [
  { icon: '🔍', text: 'Capturing screenshots...' },
  { icon: '🎨', text: 'Analysing colour palette...' },
  { icon: '🖋️', text: 'Identifying typography...' },
  { icon: '📐', text: 'Reading layout patterns...' },
  { icon: '✨', text: 'Synthesising references...' },
  { icon: '📝', text: 'Writing design documentation...' },
];

// Show:
// - Current stage with icon (animated transition between stages)
// - Elapsed time: "12 seconds"
// - Pulsing progress bar (indeterminate, not percentage-based)
// - The primary URL being processed (truncated)
// - Any reference count: "Analysing 2 additional references"
// - Estimated time remaining: "Usually takes 15-40 seconds"
// - Cancel button (calls DELETE /api/design-extract/:id if cancellation is suppo
```

B6 — DesignTokenEditor Component

The live-editing surface after extraction completes. This is the “tweak before download” experience.

Tab structure:

[Colors] [Typography] [Spacing & Shape] [Preview]

Colors tab:

```
// Grid of color swatches, 4 per row on desktop, 3 on mobile
// Each swatch:
//   - Large color circle (40px)
//   - Semantic label below (e.g. "Primary", "Background")
//   - Hex value below label
//   - Click swatch → opens ColorSwatchEditor popover
//
// ColorSwatchEditor popover:
//   - Native <input type="color"> (large, 100% width)
//   - Hex text input (with validation)
//   - "Lighten" and "Darken" quick buttons (±10% lightness)
//   - Live preview: shows text on this background, this text on white
//   - Contrast ratio display (WCAG AA/AAA pass/fail indicator)
//   - Change propagates immediately to DesignMdPreview
```

Typography tab:

```
// For each font role (Display, Body, Mono):
//   - Font family name (editable text input)
//   - "Load from Google Fonts" button → opens font search modal
//   - Live preview: show actual text rendered in that font (via @import or link)
//   - Preview text: editable ("The quick brown fox...")
//   - Weight selector: checkboxes for available weights
//   - Size scale: base size input (px/rem) + preview of the scale
//
// Google Fonts search modal:
//   - Debounced search (300ms) → fetch https://fonts.googleapis.com/...
//   - Results grid with preview
//   - Click to select → updates font family name + injects CSS link
```

Spacing & Shape tab:

```
// Spacing scale:
//   - Visual ruler showing the scale
//   - Base unit input (4 or 8, default 8)
//   - Scale values shown as labeled boxes with editable px values
//
// Border radius:
//   - 4 slots: sm / md / lg / full
//   - Visual preview of a rounded rectangle for each
//   - Slider input (0-24px, full = 9999)
```

```
//  
// Shadows:  
//   - List of shadow strings (editable textarea per shadow)  
//   - Visual preview of card with each shadow applied
```

Preview tab:

```
// Rendered DesignMdPreview (see B7)
```

B7 — DesignMdPreview Component

```
// Renders the current design.md output as:  
// Tab 1: "Design Doc" — rendered markdown (use react-markdown or similar)  
//   - YAML front matter shown as a styled token table  
//   - Sections rendered with proper headings  
//   - Color swatches inline in the Colors section  
//   - Font previews inline in the Typography section  
//  
// Tab 2: "tailwind.config.ts" — syntax-highlighted code block  
// Tab 3: "tokens.css" — syntax-highlighted code block  
// Tab 4: "design-tokens.json" — syntax-highlighted code block  
//  
// "Copy" button on each code tab  
// Regenerates automatically when tokens are edited (500ms debounce)
```

B8 — ExportPanel Component

```
// Sticky panel at bottom of left column (or fixed bottom on mobile)  
//  
// Primary actions (row):  
//   [↓ design.md] [↓ tailwind.config.ts] [↓ tokens.css] [↓ All files (.zip)]  
//  
// Secondary action:  
//   [Apply to Current Project ▼] — dropdown to select which project  
//   On confirm: calls POST /api/design-extract/:id/apply-to-project  
//   Shows success toast: "Design tokens applied to [Project Name]"  
//  
// Save to history:  
//   [★ Save extraction] — saves with a custom name  
//  
// Share:
```

```
// [🔗 Copy link] - copies /design-extract/:id URL to clipboard
```

B9 — ExtractionHistory Component

```
// Sidebar or collapsible panel showing past extractions
// Each row:
//   - Primary color swatch (first extracted color)
//   - Primary URL (truncated, favicon)
//   - Date (relative: "2 hours ago")
//   - Status badge (complete / failed)
//   - Click → navigates to /design-extract/:id
//
// Load from: GET /api/design-extract?limit=20
// Show max 10, "View all" link
// Empty state: "Your past extractions appear here"
```

B10 — Static Frontend Fallback (GitHub Pages mode)

File: artifacts/www-studio/src/lib/designExtractClient.ts (new file)

For GitHub Pages deployment (no api-server), implement the extraction pipeline in the browser:

```
export async function extractDesignInBrowser(params: ExtractionParams): Promise<D
  // Get Gemini key from localStorage (same as AiChatWidget.tsx)
  const config = getGeminiConfig(); // existing utility

  // Cannot take screenshots in browser (CORS + no headless browser)
  // Instead: prompt the user that screenshot capability requires the API server
  // Fall through to: accept image uploads only (no URL screenshots)

  // Build prompt using only uploaded images + annotations
  const prompt = buildBrowserExtractionPrompt(params);

  // Call Gemini vision API directly
  const response = await callGeminiVision(config.key, prompt);

  // Parse response into ExtractedTokens
  // Generate output files from tokens
  // Store in localStorage (no DB available)

  return extraction;
```

}

Important: In browser mode, show a banner:

"Running in browser mode — URL screenshots are not available. Upload reference images instead, or run the API server for full URL extraction."

This keeps the tool useful even without the backend.

Phase B checkpoint — verify before continuing:

- `/design-extract` loads and renders correctly on desktop and mobile
 - Primary URL input validates correctly (rejects non-URLs, non-http)
 - `+` button opens reference panel; images upload and preview correctly
 - Annotation helper chips append text to annotation field
 - Submit triggers extraction and transitions to progress view
 - Polling works: page auto-updates when extraction completes
 - DesignTokenEditor: editing a color updates DesignMdPreview in real time
 - All 4 download buttons produce correct files
 - "Apply to Current Project" works end-to-end
 - Page works in static mode (no api-server) with image uploads
-

Phase C: UI Polish & Interaction Quality

Branch checkpoint: `feature/design-intel/phase-c`

C1 — Design Language for the Page

The Design Extract page is itself a showcase of design taste. It should look better than `design-extractor.com`. Apply these intentional choices:

Color system:

- Background: near-black `#0a0a0b` (not pure black)
- Surface: `#111113`

- Surface elevated: #18181b
- Border: #27272a
- Primary accent: a single electric blue #3b82f6 (or derive from your wellness palette)
- Success: #22c55e
- Text primary: #fafafa
- Text muted: #71717a

Typography:

- Display: use `Inter Variable` at weight 600–700, letter-spacing -0.02em for headings
- Body: `Inter` weight 400, line-height 1.6
- Mono: `JetBrains Mono` or `Fira Code` for all code previews

Motion:

- Reference items slide in with a stagger (50ms between each)
- Progress stage transitions: fade + translateY(8px)
- Color swatch popover: scale(0.95) → scale(1) with 150ms ease-out
- Token editor tabs: animated underline indicator
- No full-page transitions — keep it snappy

Signature design element:

- The extraction progress screen shows a live animated “design DNA” visualization: 5–6 horizontal bars, each representing a design dimension (color, type, layout, spacing, animation), that animate from empty to filled as each stage completes. Pure CSS, no library needed.

C2 — Mobile Experience

All interactions must work on iOS Safari and Android Chrome.

- Reference upload: use `<input type="file" accept="image/*" capture="environment">` to allow camera on mobile
- Annotation chips: large enough tap targets (min 44px height)

- Color picker: native `<input type="color">` works on iOS
 - Download: handle iOS Safari's behavior (no auto-download) — show a “Long press to save” hint or open in new tab
 - Extraction history: swipe to delete (touch event handlers)
 - Token editor tabs: horizontally scrollable on narrow screens
-

C3 — Error States & Empty States

Every possible failure needs a clear, actionable UI:

URL fetch failures:

```
"Couldn't reach [url]. The site may block automated access.  
Try uploading a screenshot instead."  
[Upload Screenshot button]
```

LLM failure:

```
"Design extraction failed — the AI couldn't analyse this site.  
Common causes: very dynamic SPAs, login-required pages.  
[Try again] [Upload screenshot manually]"
```

Screenshot service unavailable:

```
"Screenshot capture isn't configured.  
Upload reference images directly, or add SCREENSHOTONE_KEY to your environment."
```

Empty history:

```
"Extract your first design →  
Paste any public website URL above to get started."
```

Partial extraction (some refs failed):

- Still show results with a warning banner: “2 of 3 references were analysed successfully”
 - Show which reference failed and why
-

C4 — Annotation UX Improvements

Annotation intelligence — show the parsed intent:

```
// After user types an annotation and blurs the field:
// Show a small pill below: "📐 Layout influence" or "🎨 Color influence"
// This confirms the system understood their note
// Derived from parseAnnotationIntent()
```

Conflict detection:

```
// If two references have conflicting intents for the same section:
// Show a warning: "Both references target typography — the last one will take pr
//                               Reorder to control which wins."
// Drag-to-reorder is the resolution mechanism
```

Smart default annotations:

```
// When user adds a URL as 2nd reference (no annotation yet):
// Auto-suggest: "Mix with primary" (pre-filled, editable)
// When user adds a URL as 3rd+ reference:
// Auto-suggest: "Use for [most unique section detected]"
```

C5 — Google Fonts Integration

File: artifacts/www-studio/src/lib/googleFonts.ts

```
// Font search: https://www.googleapis.com/webfonts/v1/webfonts?key=...&sort=popu
// Or use the subset API for typeface previews:
//   https://fonts.googleapis.com/css2?family=FONT_NAME:wght@400;700&display=swap
//
// No API key needed for CSS import endpoint
// Cache font list in localStorage (stale-while-revalidate, 24h TTL)
//
// Font preview injection:
//   Dynamically create/update <link> tags in document head
//   Remove when font is deselected (cleanup refs)
//
// Popular fonts to pre-seed (no API call needed):
//   Display: Inter, Plus Jakarta Sans, DM Sans, Syne, Cabinet Grotesk, Satoshi
//   Body: Inter, DM Sans, Nunito, Lato, Source Serif 4
//   Mono: JetBrains Mono, Fira Code, IBM Plex Mono
```

Phase C checkpoint — verify before continuing:

- Page looks polished and intentional — not generic
 - Annotation intent pills appear correctly after blur
 - Google Fonts search works and font previews render
 - Mobile: file upload and camera work on iOS
 - All error states display with correct copy and actions
 - Extraction DNA animation plays during processing
-

Phase D: RAG Integration & Project Intelligence

Branch checkpoint: `feature/design-intel/phase-d`

D1 — Auto-ingest Extraction into RAG Knowledge Base

When an extraction completes with status 'complete', automatically:

1. Chunk the `outputDesignMd` into sections (one chunk per `##` heading)
2. Add each chunk to the `knowledge_chunks` table (from Phase 1 of the main roadmap)
3. Mark the extraction `savedToKb: true`
4. The RAG system now knows this project's design language

Chunk metadata to store:

```
{
  "source": "design_extraction",
  "extractionId": "uuid",
  "projectId": "uuid",
  "section": "Colors",
  "primaryUrl": "https://example.com"
}
```

This means when the user later asks the AI editor “what’s our primary color?” or “match this button to our brand”, the AI has the full design system in context.

D2 — Design Context in AI Chat

Update `artifacts/api-server/src/routes/chat.ts` and `SceneChat.tsx` :

When a project has an associated design extraction, automatically include the first 500 chars of `outputDesignMd` in the system context of every AI call for that project.

```
// In system prompt:
const designContext = project.activeDesignExtraction
  ? `\\n\\nThis project uses the following design system:\\n${truncate(project.activ
: '';
```

Add a `activeDesignExtractionId` foreign key to the `projects` table (nullable). When "Apply to Project" is called, set this field.

D3 — Design Diff / Version History

Each time `PATCH /api/design-extract/:id/tokens` is called:

- Store a snapshot of the previous `extractedTokens` in a `design_extraction_history` JSONB array on the row (cap at 20 snapshots)
 - Frontend: show "Undo changes" button that pops the last snapshot
 - Show timestamp of last manual edit
-

D4 — Extraction Gallery (Public)

Add a `/design-extract/gallery` page showing public extractions.

When saving an extraction, users can opt-in: "Share publicly"

- Shows `primaryUrl` , primary brand color, primary font name
- Links to view the extraction (read-only)
- Useful as inspiration: "See how our community extracted Stripe, Linear, Airbnb..."

This mirrors the Scene gallery pattern already built in Phase 7.

Phase D checkpoint — verify before continuing:

- Completing an extraction auto-ingests into RAG KB
- AI chat in editor references design tokens correctly

- Undo changes works (pops history snapshot)
 - Public gallery shows shared extractions
-

Phase E: Additional Enhancements (Bonus Features)

These are the “make it legendary” items beyond the core spec.

E1 — Batch URL Extraction

```
POST /api/design-extract/batch
Body: { urls: string[] } // up to 5 URLs
```

Runs extractions in parallel, returns array of extraction IDs. Frontend: “Extract multiple sites” mode — shows 5 URL inputs. Use case: “Compare Stripe, Linear, and Notion — then synthesise the best of all three.”

E2 — Design Comparison Mode

When 2+ extractions exist, enable a side-by-side comparison view:

- `/design-extract/compare?ids=uuid1,uuid2`
 - Two-column layout showing token differences highlighted
 - “Merge best of both” button → creates a new hybrid extraction by asking the LLM to intelligently combine them
-

E3 — Figma Token Import

```
// In ExportPanel, add:
[↑ Import Figma Tokens]
// Accepts a Figma Token Studio JSON export
// Converts to ExtractedTokens format
// Populates the editor as if it were an extraction result
// Same editing/download experience, different input source
```

File: `artifacts/www-studio/src/lib/figmaTokenImporter.ts`

E4 — CSS Paste / Inspect Import

In the Reference Upload panel, add a third tab: “CSS / Code”

```
// User pastes CSS variables or a Tailwind config snippet
// System parses it into ExtractedTokens
// Example valid inputs:
//   :root { --color-primary: #3b82f6; --font-display: 'Inter'; }
//   OR a tailwind.config.ts extend block
// Converts to ExtractedTokens and merges with the extraction
```

Parsing utility: `artifacts/www-studio/src/lib/cssTokenParser.ts`

E5 — AI “Taste Critique” Mode

After extraction, offer a “Critique” button:

```
POST /api/design-extract/:id/critique
```

Asks the LLM (with your taste KB from the RAG system):

- “Rate this design system 1–10 on: distinctiveness, contrast, type quality, colour harmony”
 - Returns scores + specific improvement suggestions
 - Suggested improvements are actionable: “Increase heading font weight to 700 for more authority”
 - Each suggestion has a “Apply” button that executes the change
-

E6 — Colour Harmony Generator

In the ColorSwatchEditor, add a “Generate Harmony” button:

Given the current primary color, auto-generate:

- Complementary colour
- Analogous pair
- A neutral (derived 95% lightness version)
- A surface colour (derived 8% lightness version)

E7 — Design Token Linter

After extraction, run a fast local lint pass:

```
function lintDesignTokens(tokens: ExtractedTokens): LintWarning[] {
  const warnings = [];

  // Check contrast ratios (WCAG AA = 4.5:1 for normal text)
  if (contrastRatio(tokens.colors.text, tokens.colors.background) < 4.5) {
    warnings.push({ severity: 'error', message: 'Text/background contrast fails W
  }

  // Check for too many fonts (> 2 is a smell)
  const fontCount = [tokens.typography.displayFont, tokens.typography.bodyFont, t
    .filter(Boolean).filter((v, i, a) => a.indexOf(v) === i).length;
  if (fontCount > 3) {
    warnings.push({ severity: 'warning', message: `${fontCount} distinct fonts de
  }

  // Check for near-duplicate colors (within 5% HSL distance)
  // Check for missing semantic colors (no muted/border defined)
  // etc.
}
```

Show lint results as inline warnings in the token editor, with one-click fixes where possible.

E8 — Export to Claude Artifacts Format

Add a special export option: “Export for Claude”

Produces a prompt-ready text block:

[Copy to clipboard]

When building UI for this project, follow this design system:

```
**Primary color:** #3b82f6
**Background:** #0a0a0b
**Display font:** Inter Variable (weight 600–700)
**Body font:** Inter (weight 400)
**Border radius:** 8px (md), 4px (sm), 16px (lg)
```

[Full design.md content follows...]

This is the “drop into Claude/Cursor” use case, making www-studio a design system companion for any AI coding tool.

Phase F: Testing & Quality

Branch checkpoint: `feature/design-intel/phase-f`

F1 — Unit Tests

File locations:

```
artifacts/api-server/src/__tests__/designExtract.test.ts
artifacts/api-server/src/__tests__/intentParser.test.ts
artifacts/api-server/src/__tests__/designPrompts.test.ts
artifacts/www-studio/src/__tests__/ColorSwatchEditor.test.tsx
artifacts/www-studio/src/__tests__/DesignTokenEditor.test.tsx
```

intentParser.test.ts — Cover all annotation patterns:

```
test('detects typography intent', () => {
  expect(parseAnnotationIntent('I love this font')).toMatchObject({
    intent: 'typography',
    sections: ['typography'],
  });
});
test('detects mobile layout intent', () => {
  expect(parseAnnotationIntent('Use mobile layout from this')).toMatchObject({
    intent: 'mobile-layout',
  });
});
// etc. — at least 12 annotation pattern tests
```

designPrompts.test.ts — Test output file generation:

```
test('buildDesignMdFromTokens produces all 8 sections', () => {
  const md = buildDesignMdFromTokens(mockTokens, 'https://stripe.com');
  expect(md).toContain('## Colors');
  expect(md).toContain('## Typography');
  expect(md).toContain('## Layout');
  // etc.
```

```
});  
test('buildTailwindConfigFromTokens is valid TypeScript', ...);  
test('buildTokensCssFromTokens produces valid CSS vars', ...);
```

colorHarmony.test.ts:

```
test('generates 4 harmonious colours from primary', ...);  
test('contrast ratio calculation matches WCAG formula', ...);
```

F2 — Integration Test: Full Pipeline

Write a test that exercises the full extraction pipeline end-to-end using a mock LLM response:

```
// Mock visionComplete() to return a fixed JSON token set  
// Verify: DB row created → pipeline runs → DB updated with complete status → all  
// Verify: PATCH /tokens updates DB and regenerates outputs  
// Verify: GET /download/design-md returns correct content-type  
// Verify: GET /download/zip returns a valid zip with 4 files
```

F3 — Cross-Browser Manual QA Checklist

Before merging to main:

```
[ ] Chrome 125+ desktop: full flow works  
[ ] Safari 17+ desktop: download works (check blob URL handling)  
[ ] Firefox 126+ desktop: color picker works  
[ ] iOS Safari 17: file upload works, download shows share sheet  
[ ] Android Chrome 125: file upload works, camera option appears  
[ ] iPad Safari: two-column layout renders correctly  
[ ] PWA (installed): page accessible offline shows "Extraction requires network"
```

Implementation Order for the Agent

Follow this exact sequence. Do not skip ahead or combine phases.

1. Phase A1: Schema → push migration → verify table exists
2. Phase A2: Screenshot service → test with a real URL

3. Phase A3: Intent parser → write tests first, then implement
4. Phase A4: Design prompts → implement buildDesignMdFromTokens first and test ou
5. Phase A5: API route → implement with mock LLM first, then real calls
6. Phase A6: OpenAPI spec → run codegen
7. Phase A7: Rate limiter registration
- Phase A checkpoint ---
8. Phase B1: Route registration
9. Phase B3: Page shell with state machine
10. Phase B4: Input component (no backend calls yet – use mock data)
11. Phase B5: Progress component
12. Phase B6: Token editor (start with colors tab only)
13. Phase B7: Preview component
14. Phase B8: Export panel
15. Phase B9: History component
16. Phase B10: Static/browser fallback
- Phase B checkpoint ---
17. Phase C1: Design language pass (style everything)
18. Phase C2: Mobile optimisation pass
19. Phase C3: Error states
20. Phase C4: Annotation UX
21. Phase C5: Google Fonts
- Phase C checkpoint ---
22. Phase D1: RAG auto-ingest
23. Phase D2: Design context in AI chat
24. Phase D3: Token diff history
25. Phase D4: Public gallery
- Phase D checkpoint ---
26. Phase E1-E8: Pick the ones that excite you most, implement in any order
- Bonus phase ---
27. Phase F1: Unit tests
28. Phase F2: Integration test
29. Phase F3: Manual QA
- Merge to main ---

Environment Variables to Add

Document these in `.env.example` and `README.md`:

```
# Design Extract – Screenshot Service
SCREENSHOTONE_KEY=          # Optional: https://screenshotone.com (free tier: 100/
PLAYWRIGHT_ENDPOINT=       # Optional: self-hosted Playwright API endpoint

# Design Extract – Google Fonts (frontend)
VITE_GOOGLE_FONTS_KEY=      # Optional: for searching all fonts (not needed for CS
```

Files Changed / Created Summary

NEW FILES:

packages/db/src/schema.ts	[modified: add table]
artifacts/api-server/src/lib/screenshot.ts	
artifacts/api-server/src/lib/intentParser.ts	
artifacts/api-server/src/lib/designPrompts.ts	
artifacts/api-server/src/lib/colorHarmony.ts	
artifacts/api-server/src/routes/design-extract.ts	
artifacts/api-server/src/__tests__/designExtract.test.ts	
artifacts/api-server/src/__tests__/intentParser.test.ts	
artifacts/api-server/src/__tests__/designPrompts.test.ts	
artifacts/www-studio/src/pages/DesignExtractPage.tsx	
artifacts/www-studio/src/lib/designExtractClient.ts	
artifacts/www-studio/src/lib/googleFonts.ts	
artifacts/www-studio/src/lib/cssTokenParser.ts	
artifacts/www-studio/src/lib/figmaTokenImporter.ts	
artifacts/www-studio/src/components/design-extract/DesignExtractInput.tsx	
artifacts/www-studio/src/components/design-extract/ReferenceItem.tsx	
artifacts/www-studio/src/components/design-extract/ReferenceUploadPanel.tsx	
artifacts/www-studio/src/components/design-extract/ExtractionProgress.tsx	
artifacts/www-studio/src/components/design-extract/DesignTokenEditor.tsx	
artifacts/www-studio/src/components/design-extract/ColorSwatchEditor.tsx	
artifacts/www-studio/src/components/design-extract/TypographyEditor.tsx	
artifacts/www-studio/src/components/design-extract/SpacingEditor.tsx	
artifacts/www-studio/src/components/design-extract/DesignMdPreview.tsx	
artifacts/www-studio/src/components/design-extract/ExportPanel.tsx	
artifacts/www-studio/src/components/design-extract/ExtractionHistory.tsx	

MODIFIED FILES:

artifacts/api-server/src/index.ts	[register route + rat
artifacts/api-server/src/routes/screenshot-to-code.ts	[refactor to use lib/
artifacts/api-server/src/routes/chat.ts	[inject design contex
artifacts/api-spec/openapi.yaml	[add design-extract p
artifacts/www-studio/src/App.tsx	[add route]
artifacts/www-studio/src/components/Navbar.tsx	[add nav link]
artifacts/www-studio/src/components/SceneChat.tsx	[inject design contex

AFTER PHASE D:

packages/db/src/schema.ts	[add activeDesignExtr
artifacts/api-server/src/routes/projects.ts	[return activeDesignE

Key Conventions to Maintain

These conventions are from `agent_notes.md` and must not be broken:

1. **Always** `String(req.params.id)` before Drizzle `eq()` — extraction IDs are UUIDs but params may be typed as `string | string[]`
2. **JSON fields** — `references`, `extractedTokens` are stored as JSONB but always `JSON.parse()` on read; always `JSON.stringify()` on write
3. **LLM calls** — always import from `artifacts/api-server/src/lib/llm.ts` — never create OpenAI client inline
4. `getOrGuestUserId(req)` — use for all user ID lookups
5. **Rate limiter** — apply AI limiter to all LLM-calling endpoints before registration
6. **Run after OpenAPI changes:** `pnpm --filter @workspace/api-client-react run codegen`
7. **After schema changes:** `pnpm --filter @workspace/db run push`
8. **After every phase:** `pnpm run typecheck && pnpm run build` — zero errors before moving on